Sprawozdanie z laboratorium 2

Temat: Metoda najmniejszych kwadratów

Autorzy: Kacper Hawro, Jan Ślosarczyk
Data: 25.03.2026

1. Treść zadań

Zadanie 1. Metoda najmniejszych kwadratów.
Celem zadania jest zastosowanie metody najmniejszych kwadratów do predykcji roku wydania utworów muzycznych.

- Do rozwiązania problemu wykorzystamy zbiór danych YearPredictionMSD.txt.
- Zbiór opisuje 515345 utworów. Pierwsze $n = 463715$ przykładów tworzy zbiór treningowy. Pozostałe 51 630 przykładów tworzy zbiór testowy.
- Pierwsza kolumna w tym zbiorze opisuje rok wydania utworu.
- Pozostałe kolumny tworzą $m = 90$ cech utworów: 12 cech dotyczy średniej barwy, pozostałe 78 cech dotyczy kowariancji barwy.

(a) Otwórz zbiór YearPredictionMSD.txt, używając funkcji np.loadtxt z biblioteki numpy lub pd.read_csv z biblioteki pandas.

(b) Podziel zbiór na zbiór treningowy (pierwsze 463 715 przykładów) oraz zbiór testowy (pozostałe 51 630 przykładów).

(c) Stwórz histogram i wykres kolumny reprezentującej rok wydania utworu przy pomocy funkcji hist oraz plot. W przypadku wykresu posortuj wartości kolumny od najmniejszej do największej. Pamiętaj o podpisaniu osi i wykresów.

(d) Stwórz reprezentacje danych zawartych w obu zbiorach dla metody najmniejszych kwadratów (2 macierze). Stwórz kolejne reprezentacje danych zawartych w obu zbiorach dla metody najmniejszych kwadratów, w których dane (cechy) są znormalizowane metodą min-max scaling do przedziału [0,1]: (2 macierze): $f_{\text{scaled}} = \frac{f - f_{\min}}{f_{\max} - f_{\min}}$ gdzie $f_{\min}$ i $f_{\max}$ to minimalna i maksymalna wartość, jaką przyjmuje dana cecha w zbiorze treningowym. Analogicznie należy przeskalować wektory reprezentujące rok wydania utworu: $y_{\text{scaled}} = \frac{y - y_{\min}}{y_{\max} - y_{\min}}$

(e) Znajdź wagi dla metody najmniejszych kwadratów przy pomocy macierzy A zbudowanej na podstawie zbioru treningowego. Potrzebny będzie także wektor y zawierający rok wydania dla każdego utworu ze zbioru treningowego. Uwaga. Problem najmniejszych kwadratów rozwiąż, stosując równanie normalne. Rozwiązując równanie normalne należy użyć funkcji np.linalg.solve, unikając obliczania odwrotności macierzy funkcją scipy.linalg.pinv.

(f) Znajdź odrębny zbiór wag dla reprezentacji nieznormalizowanej, używając funkcji scipy.linalg.lstsq (która stosuje rozkład SVD do rozwiązania problemu) oraz zbiór wag dla zregularyzowanej reprezentacji nieznormalizowanej, rozwiązując równanie normalne oraz stosując $\lambda = 0.01$

(g) Oblicz współczynniki uwarunkowania macierzy, $\text{cond}(A^T A)$, wyznaczone na zbiorze treningowym, osobno dla reprezentacji nieznormalizowanej oraz znormalizowanej. Pomocna będzie funkcja np.cond. Jaki wpływ ma współczynnik uwarunkowania na liczbę cyfr znaczących w reprezentacji wag?

(h) Sprawdź jak dobrze otrzymane wagi przewidują rok wydania utworu. W tym celu pomnóż reprezentację zbioru testowgo oraz wyliczony wektor wag dla reprezentacji nieznormalizowanej. Następnie powtórz odpowiednie mnożenie dla reprezentacji znormalizowanej. W tym ostatnim przypadku otrzymany wektor z predykcjami należy poddać operacji odwrotnej do normalizacji min-max: $t_{\text{unscaled}} = \tilde{y} * (y_{\max} - y_{\min}) + y_{\min}$ Jako miarę błędu przyjmij średni błąd bezwzględny (ang. mean absolute error).

(i) Wylicz ponownie wektor wag, tym razem na sposób inkrementacyjny, korzystając z filtru Kalmana. Podziel nieznormalizowaną macierz A reprezentującą zbiór treningowy na macierze A_1, A_2, A_3, A_4, A_5, gdzie macierze A_1, A_2, A_3, A_4 zawierają po 100000 wierszy, macierz A_5 zawiera ostatnie 63715 wierszy. Po każdej iteracji wylicz na zbiorze testowym wektor z predykcjami $\hat{y}$ a na tej podstawie średni błąd bezwzględny.

2. Argumentacja rozwiązania zadań, fragmenty algorytmu, kodu ilustrującego rozwiązanie

Do analizy i operacji macierzowych wykorzystano biblioteki `pandas`, `numpy` oraz `scipy.linalg`.

(a, b, c) Wczytanie, podział i wizualizacja danych

Zbiór danych wczytano przy użyciu funkcji `pd.read_csv`, po czym wydzielono kolumnę etykiet (rok wydania) oraz cechy dla pierwszych 463 715 przykładów (zbiór treningowy) oraz pozostałych 51 630 (zbiór testowy). Na podstawie etykiet narysowano histogram (`plt.hist`) oraz wykres rosnący wydań w czasie (`plt.plot`).

```
# Load data
data = pd.read_csv("YearPredictionMSD.txt",header=None)

# Split into training and testing sets
training_set = data.iloc[:463715]
testing_set = data.iloc[463715:]

# Extract the first column (release year) for the entire dataset
release_year = data[0]

plt.title("Histogram")
plt.hist(release_year, bins=50, color = 'lightblue', edgecolor='black')
plt.xlabel("Release year")
plt.ylabel("Count")
plt.show()

release_year_sorted = release_year.sort_values().reset_index(drop=True)

plt.plot(release_year_sorted.values, release_year_sorted.index, color='purple')
plt.grid(True)
plt.title("Years of song releases in order")
plt.ylabel("Song number (after sorting)")
plt.xlabel("Release year")
plt.show()
```

(d) Normalizacja danych

Przygotowano wariant danych nieznormalizowanych (macierz A z dodaną kolumną jedynek dla wyrazu wolnego) oraz znormalizowanych metodą Min-Max do przedziału $[0, 1]$. Wartości ekstremalne min/max zostały wyliczone wyłącznie na zbiorze treningowym i wykorzystane do przeskalowania zbioru testowego.

```
# Training and testing data (unscaled)
y_train = training_set.iloc[:, 0].values
X_train = training_set.iloc[:, 1:].values

y_test = testing_set.iloc[:, 0].values
X_test = testing_set.iloc[:, 1:].values

# Getting Min-Max values
X_min = X_train.min(axis=0)
X_max = X_train.max(axis=0)

# Scaling (X)
X_train_scaled = (X_train - X_min) / (X_max - X_min)
X_test_scaled = (X_test - X_min) / (X_max - X_min)

# Calculating min and max values
y_min = y_train.min()
y_max = y_train.max()

# Scaling (y)
y_train_scaled = (y_train - y_min) / (y_max - y_min)
y_test_scaled = (y_test - y_min) / (y_max - y_min)
```

(e, f) Równanie normalne, SVD i regularyzacja

Zamiast jawnie obliczać odwrotność macierzy funkcją `pinv`, zastosowano bezpieczne numerycznie algorytmy. Podstawowe wagi wyliczono przez rozwiązanie układu $A^T Aw = A^T y$ z użyciem `np.linalg.solve`.

Kolejne zestawy wag wyliczono algorytmem SVD z biblioteki scipy (`la.lstsq`) oraz dodając element kary λI (gdzie $\lambda = 0.01$) dla regularyzacji, co pomaga rozwiązać problem zbyt dużego powiązania się ze sobą cech oraz uniknąć błędów numerycznych.

```
# Unscaled data

ones_train = np.ones((X_train.shape[0], 1))
A_train = np.hstack((ones_train, X_train))

A_T = A_train.T
left_side = A_T @ A_train
right_side = A_T @ y_train

w_unscaled = np.linalg.solve(left_side, right_side)

# 1. SVD method from the SciPy library
w_svd, residuals, rank, s = la.lstsq(A_train, y_train)
print("Weight vector shape (SVD):", w_svd.shape)

# 2. Normal equation with regularization (lambda = 0.01)
lam = 0.01
I = np.eye(A_train.shape[1])
I[0, 0] = 0
```

```
left_side_reg = left_side + (lam * I)
w_reg = np.linalg.solve(left_side_reg, right_side)
print("Weight vector shape (regularized):", w_reg.shape)
```

(g) Uwarunkowanie macierzy i utrata precyzji

W tym kroku obliczono współczynnik uwarunkowania, na zbiorze treningowym zarówno dla reprezentacji nieznormalizowanej, jak i znormalizowanej. Współczynnik uwarunkowania informuje nas, jak bardzo rozwiązanie układu równań liniowych (wektor wag) jest wrażliwe na drobne błędy numeryczne.

Wpływ na cyfry znaczące: Ogólna zasada w analizie numerycznej mówi, że jeśli współczynnik uwarunkowania wynosi 10^k, to podczas obliczeń tracimy około k cyfr znaczących precyzji. Bardziej formalnie, jeśli współczynnik uwarunkowania wynosi x, liczbę utraconych dziesiętnych cyfr precyzji można oszacować za pomocą wzoru:

$$\text{Utracone cyfry} \approx \log_{10}(x)$$

(h) Oceny modelu

Dla wszystkich modeli wyliczono predykcje na zbiorze testowym i zmierzono średni błąd bezwzględny (MAE). W przypadku danych znormalizowanych, przed wyliczeniem błędu, wyniki predykcji poddano operacji odwrotnej do normalizacji min-max.

```
# 1. Prepare test matrices (adding a column of ones, because first )
ones_test = np.ones((X_test.shape[0], 1))
A_test = np.hstack((ones_test, X_test))

ones_test_scaled = np.ones((X_test_scaled.shape[0], 1))
A_test_scaled = np.hstack((ones_test_scaled, X_test_scaled))

# 2. Predict using unscaled data
y_pred_unscaled = A_test @ w_unscaled

# 3. Predict using scaled data
y_pred_scaled = A_test_scaled @ w_scaled

# Reverse the Min-Max scaling for the scaled predictions
y_pred_scaled_reversed = y_pred_scaled * (y_max - y_min) + y_min

y_pred_svd = A_test @ w_svd

y_pred_reg = A_test @ w_reg


# 4. Calculate Mean Absolute Error (MAE)
mae_unscaled = np.mean(np.abs(y_pred_unscaled - y_test))
mae_scaled = np.mean(np.abs(y_pred_scaled_reversed - y_test))
mae_svd = np.mean(np.abs(y_pred_svd - y_test))
mae_reg = np.mean(np.abs(y_pred_reg - y_test))

print(f"Mean Absolute Error (unscaled): {mae_unscaled:.6f} years")
print(f"Mean Absolute Error (scaled): {mae_scaled:.6f} years")
print(f"Mean Absolute Error (SVD): {mae_svd:.6f} years")
print(f"Mean Absolute Error (Regularization): {mae_reg:.6f} years")
```

(i) Filtr Kalmana (podejście inkrementacyjne)

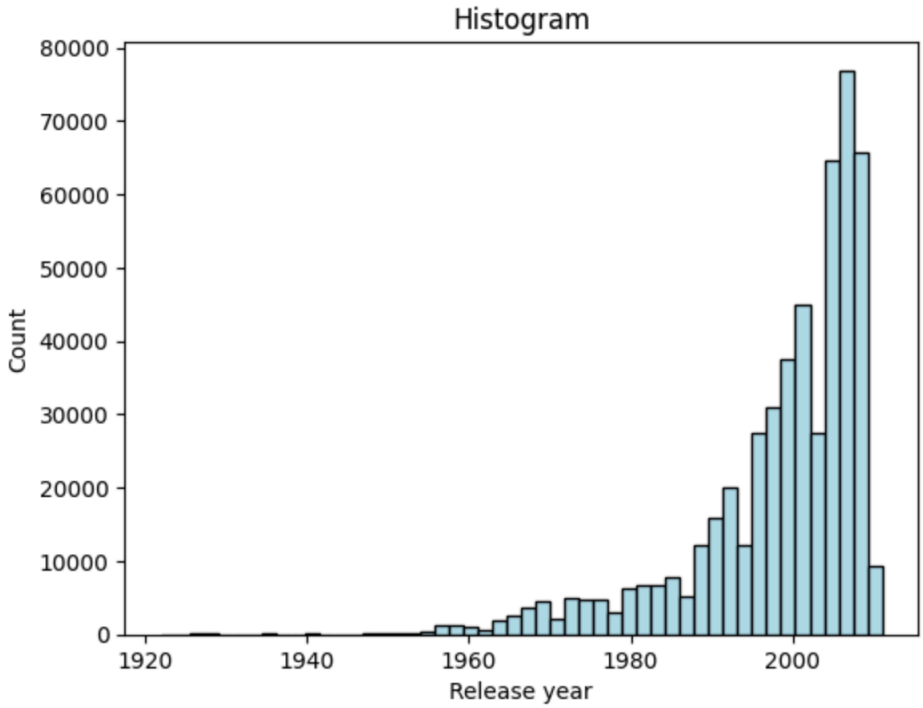
Filtr Kalmana pozwala na aktualizację modelu w miarę napływu nowych danych bez konieczności

trzymania całego zbioru w pamięci operacyjnej. Zbiór treningowy nieznormalizowany A podzielono na partie po 100 000 wierszy. Realizując to podejście strumieniowe, zastosowano algorytm krok po kroku:

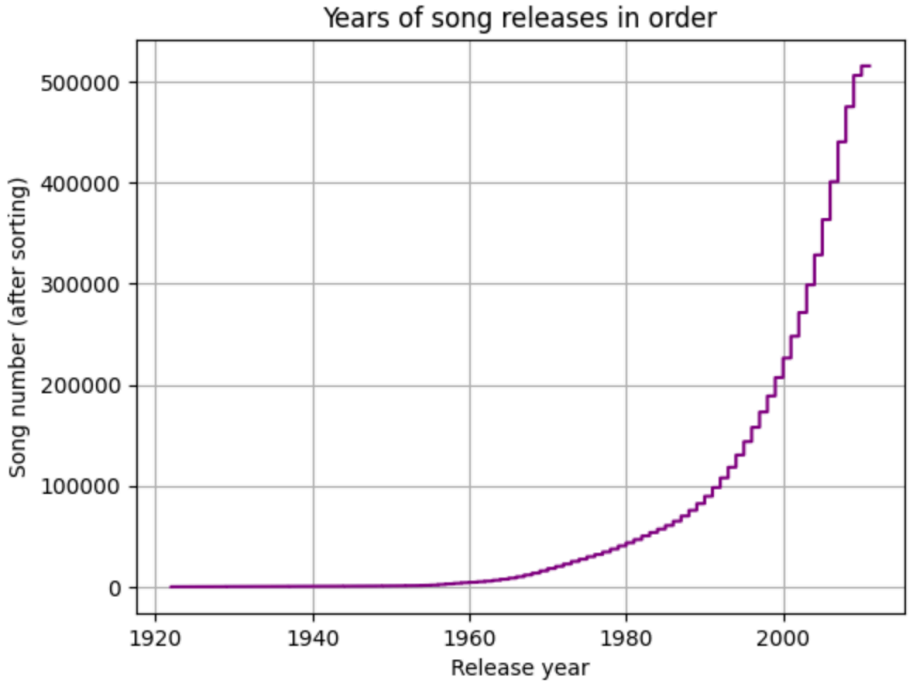
1. **Inicjalizacja:** Początkowy wektor wag w_0 oraz macierz H_0 ustawiono jako macierze zerowe.
2. **Aktualizacja macierzy H:** Dla każdej napływającej paczki danych A_j oraz y_j, macierz H (reprezentującą składnik $A^T A$) aktualizowano iteracyjnie dodając nową informację za pomocą wzoru $H_j = H_{j-1} + A_j^T A_j$.
3. **Obliczenie poprawki d_j:** Wyznaczano różnicę między rzeczywistymi etykietami a predykcją na bieżącej paczce, a następnie rozwiązywano układ równań $H_j d_j = A_j^T (y_j - A_j w_{j-1})$ w celu znalezienia wektora zmiany d_j.
4. **Aktualizacja wag w_j:** Korygowano model, dodając wyliczoną poprawkę do wektora wag z poprzedniego kroku: $w_j = w_{j-1} + d_j$.

Proces ten powtarzano w pętli dla wszystkich partii danych. W ramach eksperymentu, po zakończeniu każdego takiego kroku wyznaczano dodatkowo testowe MAE, aby zaobserwować stabilizowanie się i proces uczenia modelu.

3. Wykresy, tabele, wyniki liczbowe



Rysunek 1: Histogram kolumny roku wydania utworu



Rysunek 2: Posortowane wartości kolumny roku wydania utworu

Tabela 1. Uwarunkowanie macierzy (Zadanie g)

Rodzaj reprezentacji zbioru treningowego	$\mathrm{cond}(A^T A)$
Nieznormalizowana	$3.8737 \cdot 10^9$
Znormalizowana (min-max)	$2.1465 \cdot 10^6$

Tabela 2. Zestawienie błędu MAE na zbiorze testowym (Zadanie h)

Metoda	Błąd MAE
Równanie normalne (dane nieznormalizowane)	6.800496
Równanie normalne (dane znormalizowane, zdekodowane)	6.800496
Dekompozycja SVD (`lstsq`)	6.800496
Zregularyzowane równanie normalne ($\lambda = 0.01$)	6.800496

Tabela 3. Predykcja za pomocą Filtru Kalmana (Zadanie i)

Krok filtru	Rozmiar bloku	Błąd MAE po kroku
Po partii A_1	100 000 wierszy	6.846108
Po partii A_2	100 000 wierszy	6.806518
Po partii A_3	100 000 wierszy	6.799756
Po partii A_4	100 000 wierszy	6.801075
Po partii A_5	63 715 wierszy	6.800496

4. Wnioski, obserwacje, uwagi

- **Uwarunkowanie macierzy (Zadanie g)**: Wyniki w Tabeli 1 ukazują ogromny współczynnik uwarunkowania dla macierzy nieznormalizowanej (rząd 10^9). Skalowanie Min-Max znacząco go obniżyło (do rzędu 10^6). Złe uwarunkowanie macierzy sprawia, że wyliczone wagi stają się słabo zdeterminowane numerycznie i narażone na poważną utratę precyzji cyfr znaczących. Kluczową obserwacją teoretyczną i praktyczną jest jednak to, że złe uwarunkowanie macierzy nie zepsuło samej predykcji. Zjawisko to pokazują wyniki z Tabeli 2 – mimo różnych współczynników uwarunkowania, modele na danych znormalizowanych i nieznormalizowanych wygenerowały dokładnie taki sam błąd MAE na zbiorze testowym.

- **Zastosowanie SVD i regularyzacji (Zadanie h)**: Klasyczne równania normalne są silnie wrażliwe na uwarunkowanie macierzy głównej, przez co zaleca się unikanie bezpośredniego odwracania macierzy, a metody takie jak rozkład SVD stanowią numerycznie stabilniejszą, choć wolniejszą obliczeniowo alternatywę. Z kolei zastosowana regularyzacja dodana do równania normalnego poprawia stabilność poprzez podbicie wyznacznika od zera. W przeprowadzonym eksperymencie (Tabela 2) niezwykle interesujący jest fakt, że wszystkie metody dały co do ułamka identyczny wynik średniego błędu absolutnego (6.800496 lat). Świadczy to o tym, że dla tak ogromnego, silnie nadokreślonego zbioru danych, niewielkie bezpieczniki numeryczne (SVD czy niska wartość kary λ) modyfikują wagi zaledwie mikroskopijnie, nie wpływając na ostateczną zdolność predykcyjną modelu.

- **Skuteczność Filtru Kalmana (Zadanie i)**: Algorytm filtru Kalmana sprawdza się doskonale w sytuacjach, w których dane napływają w czasie lub są zbyt duże, by przetwarzać je w całości w pamięci operacyjnej. Analiza Tabeli 3 na żywo ukazuje proces tzw. uczenia online. Błąd MAE z najwyższej wartości początkowej (Krok 1: 6.846) sukcesywnie zbiega w dół, dopasowując się do

informacji z nowych próbek. Drobne fluktuacje, jak lokalne minimum w Kroku 3 (6.799), wynikają naturalnie ze specyfiki i zróżnicowania cech utworów w danej paczce (np. nagromadzenie utworów z podobnej epoki). Najważniejszy jest jednak finał: po przetworzeniu pełnego zbioru (Krok 5) model „strumieniowy” osiągnął absolutnie identyczny błąd predykcyjny (6.800496) co model przeliczony w sposób statyczny. Dowodzi to pełnej poprawności i skuteczności zaimplementowanego algorytmu inkrementacyjnego.